Problem 1:

Repeated Substitution:

2.) $T(n) = 5 \cdot T(\frac{n}{7}) + 3n^2$

SUB 1: $T(\frac{n}{7}) = 5 \cdot T(\frac{n/7}{7}) + 3(\frac{n}{7})^2$

$= 5 \cdot T(\frac{n}{49}) + 3 \cdot \frac{n^2}{49}$

$T(n) = 5 \cdot T[5 \cdot T(\frac{n}{49}) + 3 \cdot (\frac{n^2}{49})] + 3n^2$

$= 5 \cdot 5 \cdot T(\frac{n}{49}) + 5 \cdot 3 \cdot (\frac{n^2}{49}) + 3n^2$

$= 5^2 \cdot T(\frac{n}{49}) + 5 \cdot (\frac{3n^2}{49}) + 3n^2$

$= 5^2 \cdot T(\frac{n}{49}) + 3n^2 \cdot 5 \cdot \frac{1}{49} + 3n^2$

$T(n) = 5^2 \cdot T(\frac{n}{49}) + 3n^2 \cdot (\frac{5}{49}) + 3n^2$

SUB 2: $T(\frac{n}{49}) = 5 \cdot T(\frac{n/49}{7}) + 3 \cdot (\frac{n}{49})^2$

$= 5 \cdot T(\frac{n}{343}) + 3 \cdot (\frac{n^2}{2401})$

$T(n) = 5^2 \cdot [5 \cdot T(\frac{n}{343}) + 3 \cdot (\frac{n^2}{2401})] + 3n^2 \cdot (\frac{5}{49}) + 3n^2$

$= 5^2 \cdot 5 \cdot T(\frac{n}{343}) + 5^2 \cdot 3 \cdot (\frac{n^2}{2401}) + 3n^2 \cdot (\frac{5}{49}) + 3n^2$

$= 5^3 \cdot T(\frac{n}{343}) + 5^2 \cdot (\frac{3n^2}{2401}) + 3n^2 \cdot (\frac{5}{49}) + 3n^2$

$= 5^3 \cdot T(\frac{n}{343}) + 3n^2 \cdot (\frac{5}{49})^2 + 3n^2 \cdot (\frac{5}{49}) + 3n^2$

Pattern: $5^k \cdot T(\frac{n}{7^k}) + 3n^2 \cdot [(\frac{5}{49})^{k-1} + \dots + (\frac{5}{49})^1 + (\frac{5}{49})^0]$

$\frac{n}{7^k} = 1$

$7^k = n$

$k = \log_7(n)$

$$T(n) = \overbrace{5^k \cdot T\left(\frac{n}{7^k}\right)}^{P1} + \overbrace{3n^2 \cdot \sum \frac{5}{49}^{i}}^{P2}$$

$$5^k = 5^{\log_7 n} = n^{0.827} \leftarrow P1$$

$$3n^2 \cdot \sum \left(\frac{5}{49}\right)^i$$

$$3n^2 \cdot \text{constant} = n^2 \leftarrow P2$$

$$P1: \; n^{0.827}$$

$$P2: \; n^2 \leftarrow \text{bigger}$$

$$T(n) = O(n^2)$$

Master Method:

Formula: $T(n) = aT(n/b) + f(n)$

$T(n) = 5 \cdot T(n/7) + 3n^2$

- $a = 5$
- $b = 7$
- $d = 2$

b^d = 7^2 = 49

Case 1: a < b^d

5< 49

Since a < b^d, the rule is:

T(n) = Θ(n^d) = Θ(n^2)

Final Answer: T(n) = Θ(n^2)

Problem 2:

a. T(n) = 5T(n/2) + n^7

- a = 5, b = 2, f(n) = n^7 → d = 7
- b^d = 2^7 = 128
- a vs b^d → 5 vs 128 → 5 < 128
- a < b^d so we use rule: T(n) = Θ(n^d)
- Final answer: T(n) = Θ(n^7)

b. T(n) = 3T(n/2)+7n

- a = 3, b = 2, f(n) = 7n → d = 1
- b^d = 2^1 = 2
- a vs b^d → 3 vs 2 → 3 > 2
- a > b^d so we use rule: T(n) = Θ(n^(log_b a))
- Log $_2$(3) ≈ 1.585
- Final Answer: T(n) = Θ(n^($\log_2$3))

c. T(n) = 15T(n/3)+100

- a = 15, b = 3, f(n) = 100 = $100 \cdot n^0$ → d = 0
- b^d = 3^0 = 1
- b^d = 3^0 = a vs b^d → 15 vs 1 → 15 > 1
- a > b^d so we use rule: T(n) = Θ(n^(log_b a))
- $\log_3(15) \approx 2.465$
- Final Answer: T(n) = Θ(n^($\log_3$15))

d. T(n) = 9T(n/3)+n^2

- a = 9, b = 3, f(n) = n^2 → d = 2
- b^d = 3^2 = 9
- a vs b^d → 9 vs 9 → 9 = 9
- a = b^d so we use rule: T(n) = Θ(n^d · log n)
- Final Answer: T(n) = Θ(n^2 log n)

e. T(n) = 6T(n/2) + n^3

- a = 6, b = 2, f(n) = n^3 → d = 3
- b^d = 2^3 = 8
- a vs b^d → 6 vs 8 → 6 < 8
- a < b^d so we use rule: T(n) = Θ(n^d)
- Final Answer: T(n) = Θ(n^3)

Problem 3:

Pass 1: Sorting by the LAST character

A: GUA, LA_GUA, META, AMA, NSA, GUA,

C: SUC, VIC,

G: MAG,

I: QUI, HUI,

L: ATL, VAL, CAL, TOL, BOL,

N: SAN, SAN,

O: CHO,

Q: CAQ,

R: COR,

S: RIS, CES, CAS,

T: PUT, ANT,

U: CAU, VAU,

Y: BOY

Pass 2: Sorting by 2nd to LAST character:

A: MAG, VAL, CAL, SAN, SAN, CAQ, CAS, CAU, VAU

C: CES, CHO, COR

O: TOL, BOL, BOY

N: ANT

I: RIS, VIC

U: QUI, HUI, PUT, GUA, LA_GUA, GUA

M: AMA

L: ATL

S: SUC

T: META

Pass 3: Sorting by the 3rd to LAST character

A: ANT, AMA, ATL

B: BOL, BOY

C: CAL, CAQ, CAS, CAU, CES, CHO, COR

G: GUA, LA_GUA, GUA

H: HUI

M: MAG, META

N: NSA

P: PUT

Q: QUI

R: RIS

S: SAN, SAN, SUC

T: TOL

V: VAL, VAU, VIC

Pass 4: Sorting by the 4th to LAST character

_: LA_GUA

A: AMA, ANT, ATL

B: BOL, BOY

C: CAL, CAQ, CAS, CAU, CES, CHO, COR

G: GUA, GUA

H: HUI

M: MAG, META

N: NSA

P: PUT

Q: QUI

R: RIS

S: SAN, SAN, SUC

T: TOL

V: VAL, VAU, VIC

Pass 5: Sorting by the 5th to LAST character

A: AMA, ANT, ATL, LA_GUA

B: BOL, BOY

C: CAL, CAQ, CAS, CAU, CES, CHO, COR

G: GUA, GUA

H: HUI

M: MAG, META

N: NSA

P: PUT

Q: QUI

R: RIS

S: SAN, SAN, SUC

T: TOL

V: VAL, VAU, VIC

Pass 6: Sorting by the 6th to LAST character

L: LA_GUA

Final Sorted List:

AMA

ANT

ATL

BOL

BOY

CAQ

CAL

CAS

CAU

CES

CHO

COR

GUA

GUA

HUI

LA_GUA

MAG

META

NSA

PUT

QUI

RIS

SAN

SAN

SUC

TOL

VAL

VAU

VIC

Problem 4:

M = 7

Slot	1	1	2	3	4	5	6
Key							

KEY 62

Step 1: h1(62)

- x = (|62 - 9|) * (62 + 13)
- x = (53) * (75)
- x = 3975
-
- x = 3975 / 15
- x = 265
-
- x = 265 + 62
- x = 327

- x = 327 % 7
- x = 327 - 46*7 = 327 - 322 = 5
- The home slot is slot 5.

Step 2: Insert into slot 5

Slot	1	1	2	3	4	5	6
Key						62	

KEY 62 again

Step 1: h1(62) again

- Same key -> result at slot 5
- Slot 5 is taken, so a collision occurred

Step 2: do power(62)

- power(62) = $62^3 + 1$

- = 238328 + 1
- = 238329

Step 3: probe

- Attempt 1: (5 + 1·238329) % 7
- = 238334 % 7
- = 238334 - 34047*7
- = 238334 - 238329
- = 5
- Still slot 5

- Attempt 2: (5 + 2·238329) % 7
- = 476663 % 7
- = 476663 - 68094*7
- = 476663 - 476658
- = 5
- Still slot 5

- Attempt 3: (5 + 3·238329) % 7
- = 714992 % 7
- = 714992 - 102141*7
- = 714992 - 714987
- = 5
- Still slot 5
- 3 attempts didn't work, so resize

Step 4: Resize

- Current M = 7
- 2 * 7 = 14
- Next prime after 14 = 17
- New M = 17

Step 5: Rehash

Compute H1(62)

- x = (|62 - 9|) * (62 + 13)
- x = 53 * 75
- x = 3975
-
- x = 3975 / 15
- x = 265
-
- x = 265 + 62
- x = 327
-
- x = 327 % 17
- x = 327 - 19*17
- x = 327 - 323
- x = 4

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key					62												

Step 6: Compute H1(62) again but Collision occurred because of duplicate home slot

Step 7: Compute power(62)

- power(62) = $62^3 + 1 = 238329$
-
- Attempt 1: (4 + 1·238329) % 17
- = 238333 % 17
- = 238333 - 14019*17
- = 238333 - 238323
- = 10

Slot 10 is empty! Insert there.

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key					62						62						

KEY 69

Step 1: h1(69)

- x = (|69 - 9|) * (69 + 13)
- x = (60) * (82)
- x = 4920
-
- x = 4920 / 15
- x = 328
-
- x = 328 + 69
- x = 397
-
- x = 397 % 17
- x = 397 - 23*17
- x = 397 - 391
- x = 6

Step 2: Slot 6 is empty, insert it

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key					62		69				62						

KEY 70

Step 1: compute h1(70)

- x = (|70 - 9|) * (70 + 13)
- x = (61) * (83)
- x = 5063
-
- x = 5063 / 15
- x = 337
-
- x = 337 + 70

- x = 407
-
- x = 407 % 17
- x = 407 - 23*17
- x = 407 - 391
- x = 16

Home slot 16 is empty

Step 2: Insert to slot 16

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key					62		69				62						70

KEY 4

Step 1: compute h1(4)

- x = (|4 - 9|) * (4 + 13)
- x = (5) * (17)
- x = 85
-
- x = 85 / 15
- x = 5
-
- x = 5 + 4
- x = 9
-
- x = 9 % 17
- x = 9

Slot 9 is empty, so insert it

Step 2: insert into slot 9

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key					62		69			4	62						70

KEY 103

Step 1: compute h1(103)

- x = (|103 - 9|) * (103 + 13)
- x = (94) * (116)
- x = 10904
-
- x = 10904 / 15
- x = 726
-
- x = 726 + 103
- x = 829
-
- x = 829 % 17
- x = 829 - 48*17
- x = 829 - 816
- x = 13

Slot 13 is empty, insert it

Step 2: insert into slot 13

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key					62		69			4	62			103			70

KEY 85

Step 1: compute h1(85)

- x = (|85 - 9|) * (85 + 13)
- x = (76) * (98)
- x = 7448
-
- x = 7448 / 15
- x = 496
-
- x = 496 + 85
- x = 581
-
- x = 581 % 17
- x = 581 - 34*17
- x = 581 - 578

- x = 3

Home slot 3 is empty, insert it

Step 2: Insert into slot 3

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key				85	62		69			4	62			103			70

KEY 27

Step 1: compute h1(27)

- x = (|27 - 9|) * (27 + 13)
- x = (18) * (40)
- x = 720
-
- x = 720 / 15
- x = 48
-
- x = 48 + 27
- x = 75
-
- x = 75 % 17
- x = 75 - 4*17
- x = 75 - 68
- x = 7

Slot 7 is empty, insert it

Step 2: insert into slot 7

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key				85	62		69	27		4	62			103			70

KEY 86

Step 1: compute h1(86)

- x = (|86 - 9|) * (86 + 13)
- x = (77) * (99)

- x = 7623
-
- x = 7623 / 15
- x = 508
-
- x = 508 + 86
- x = 594
-
- x = 594 % 17
- x = 594 - 34*17
- x = 594 - 578
- x = 16

Slot 16 is NOT empty, therefore a Collision occurred

Step 2: compute power(86)

- power(86) = 86^3 + 1
- = 636056 + 1
- = 636057

Step 3: Probe

- Attempt 1: (16 + 1·636057) % 17
- = 636073 % 17
- = 636073 - 37416*17
- = 636073 - 636072
- = 1

Slot 1 is empty, insert it there

Step 4: insert at slot 1

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key		86		85	62		69	27		4	62			103			70

KEY 40

Step 1: compute h1(40)

- x = (|40 - 9|) * (40 + 13)
- x = (31) * (53)

- x = 1643
-
- x = 1643 / 15
- x = 109
-
- x = 109 + 40
- x = 149
-
- x = 149 % 17
- x = 149 - 8*17
- x = 149 - 136
- x = 13

Slot 13 is NOT empty, therefore collision occurred

Step 2: compute power(40)

- power(40) = 40^3 + 1
- = 64000 + 1
- = 64001

Step 3: probe (attempt 1)

- Attempt 1: (13 + 1·64001) % 17
- = 64014 % 17
- = 64014 - 3765*17
- = 64014 - 64005
- = 9

Slot 9 is NOT empty, therefore collision occured

Step 4: probe (attempt 2)

- Attempt 2: (13 + 2·64001) % 17
- = 128015 % 17
- = 128015 - 7530*17
- = 128015 - 128010
- = 5

Slot 5 is empty, insert it

Step 5: insert into slot 5

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key		86		85	62	40	69	27		4	62			103			70

KEY 96

Step 1: compute h1(86)

- x = (|96 - 9|) * (96 + 13)
- x = (87) * (109)
- x = 9483
-
- x = 9483 / 15
- x = 632
-
- x = 632 + 96
- x = 728
-
- x = 728 % 17
- x = 728 - 42*17
- x = 728 - 714
- x = 14

Slot 14 is empty, insert it there

Step 2: insert into slot 14

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key		86		85	62	40	69	27		4	62			103	96		70

KEY 77

Step 1: compute h1(77)

- x = (|77 - 9|) * (77 + 13)
- x = (68) * (90)
- x = 6120
-
- x = 6120 / 15
- x = 408
-

- x = 408 + 77
- x = 485
-
- x = 485 % 17
- x = 485 - 28*17
- x = 485 - 476
- x = 9

Slot 9 is NOT empty, therefore a collision occurred

Step 2: compute power(77)

- power(77) = 77^3 + 1
- = 456533 + 1
- = 456534

Step 3: probe (attempt 1)

- Attempt 1: (9 + 1·456534) % 17
- = 456543 % 17
- = 456543 - 26855*17
- = 456543 - 456535
- = 8

Slot 8 is empty, insert it

Step 4: insert into slot 8

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key		86		85	62	40	69	27	77	4	62			103	96		70

KEY 23

Step 1: compute h1(23)

- x = (|23 - 9|) * (23 + 13)
- x = (14) * (36)
- x = 504
-
- x = 504 / 15
- x = 33
-

- x = 33 + 23
- x = 56
-
- x = 56 % 17
- x = 56 - 3*17
- x = 56 - 51
- x = 5

Slot 5 is NOT empty, therefore collision occurred

Step 2: power(23)

- power(23) = 23^3 + 1
- = 12167 + 1
- = 12168

Step 3: probe (attempt 1)

- Attempt 1: (5 + 1·12168) % 17
- = 12173 % 17
- = 12173 - 716*17
- = 12173 - 12172
- = 1

Slot 1 is NOT empty, therefore collision occurred

Step 4: probe (attempt 2)

- Attempt 2: (5 + 2·12168) % 17
- = 24341 % 17
- = 24341 - 1432*17
- = 24341 - 24327
- = 14

Slot 14 is NOT empty, therefore collision occurred

Step 5: probe (attempt 3)

- Attempt 3: (5 + 3·12168) % 17
- = 36509 % 17
- = 36509 - 2147*17
- = 36509 - 36499

- = 10

Slot 10 is NOT empty, therefore a collision occurred.

Step 6: probe (attempt 4)

- Attempt 4: (5 + 4·12168) % 17
- = 48677 % 17
- = 48677 - 2863*17
- = 48677 - 48671
- = 6

Slot 6 is NOT empty, therefore a collision occurred

Step 7: probe (attempt 5)

- Attempt 5: (5 + 5·12168) % 17
- = 60845 % 17
- = 60845 - 3579*17
- = 60845 - 60843
- = 2

Slot 2 is empty, insert it

Step 8: insert in slot 2

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key		86	23	85	62	40	69	27	77	4	62			103	96		70

Final Table:

Slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Key		86	23	85	62	40	69	27	77	4	62			103	96		70

Problem 7:

Problem 5:

The time complexity is O(n·m), where n is the number of strings in the array and m is the length of the longest string. This is because the main sorting loop runs once for every character position (m times), and for each position it looks at every string in the array (n times). Since these loops are nested, we multiply them together giving us O(n·m).

The space complexity is O(n), where n is the number of strings in the array. This is because at each pass we create a HashMap that stores all n strings in buckets. The size of this HashMap grows linearly with the number of strings, so we need O(n) extra memory.

Problem 6:

The time complexity is $O(n^3 \cdot m)$, where n is the number of words in the array and m is the length of the longest word. This is because there are three nested loops that try every possible combination of three different indexes, which gives us $O(n^3)$. Inside those loops we call isPalindrome() which checks every character in the combined string, taking O(m) time. Since isPalindrome() runs inside the three nested loops we multiply them together giving us $O(n^3 \cdot m)$.

The space complexity is $O(n^3)$, where n is the number of words in the array. This is because in the worst case every possible combination of three words forms a palindrome, meaning the result list could store up to n^3 triplets. Each triplet holds 3 integers, so the total extra memory we need grows as $O(n^3)$.